

Activation Steering for Smart Contract Vulnerability Detection: A Calibration Mechanism Bounded by Pre-Training Knowledge

Abhirup Mukherjee¹, Latifur Khan¹, and Bhavani Thuraisingham¹

Abstract—Activation steering involves injecting learned direction vectors into a language model’s hidden states during inference. We present the first application of this technique to Solidity smart contract vulnerability detection, and test it on six models with parameters ranging between 7 and 34 billion. We evaluate this method for a multi-class classification task consisting of 11 classes (DASP (Decentralized Application Security Project) Top 10 vulnerabilities in addition to the SAFE category) on the SmartBugs Curated dataset. In this paper, activation steering is the only lightweight intervention technique that demonstrates improvement over a few-shot baseline. Particularly, this method achieves a 4.5 percentage point accuracy improvement for unchecked low-level calls, with a nominal p-value of 0.006 using McNemar’s test for categorical data. We further test this method using random direction controls across six models and perform variance analysis with different seeds. We also compare two vector extraction techniques: cross-domain group means and domain-matched contrastive pairs. We discover that the less accurate method yields greater improvements, and we draw the conclusion that activation steering is a confidence calibration mechanism rather than improving vulnerability reasoning. We further show a negative correlation between baseline accuracy and steering improvement across models, suggesting a pre-training knowledge boundary.

Index Terms—activation steering, smart contract vulnerability detection, large language models.

I. INTRODUCTION

Vulnerabilities in smart contracts have caused damages of over \$16 billion to date in decentralized finance (DeFi) [1]. Despite having relatively low false positive rates in detecting common vulnerability patterns, static code analysis tools such as Slither [2] offer limited support for a wide range of vulnerability types. In recent times, large language models (LLMs) have emerged as potential complementary solutions to traditional tools—for example, a fine-tuned LLM-based pipeline achieved an F1 score of 93.3% on the SmartBugs Curated dataset [3]. Nonetheless, the internal mechanisms by which LLMs represent vulnerability concepts remain opaque, and thus warrant further investigation to improve both interpretability and reliability of LLM-assisted vulnerability analysis.

Activation steering (or representation engineering) is an experimental approach that influences the behavior of large language models via manipulation of internal representations during inference [4], [5]. It is performed via the injection of a

learned directional vector into the hidden states of the model. It does not require fine-tuning or computation of gradients—just a simple vector addition during the forward pass [6]. Activation steering is the cheapest intervention method on frozen models and falls roughly between prompting (no cost) and fine-tuning (large cost) in terms of compute cost.

Li et al. [7] showed that activation steering improves vulnerability detection in C/C++, achieving an F1 score increase of up to +16% across eight models over the baseline. However, currently there exists no published work that uses this technique to detect smart contract vulnerabilities. Solidity smart contracts have highly domain-specific vulnerability categories such as reentrancy, unchecked low-level calls, and access control, that are not present in regular code—this makes detection a significantly more challenging task [8]. In this paper, while we do not aim to surpass fine-tuning, we do wish to determine the most efficient possible intervention on frozen model representations in the blockchain and smart contract domain.

We make the following contributions in this work:

- 1) This is the first application of the activation steering technique for multi-class vulnerability classification in Solidity smart contracts, with evaluation on six open-source models with parameters ranging between 7 and 34 billion.
- 2) We compare two steering vector extraction methods: cross-domain group means and domain-matched contrastive pairs, showing that steering serves to calibrate the model’s confidence and not necessarily improve its reasoning.
- 3) We systematically compare several lightweight interventions for this task: zero-shot, few-shot, and steered few-shot, and show that only steering offers improvements over few-shot prompting.
- 4) We demonstrate that steering is directionally specific and offers improvements over noise from few-shot example selection by using random direction vectors and seed variance across six models.

Sections II and III discuss the related work and methodology, while Sections IV and V present the results and discussion, respectively.

II. RELATED WORK

Our work builds on two threads in automated vulnerability detection—activation steering as an intervention mechanism at inference-time, and usage of LLMs in vulnerability detection.

¹The authors are with the Department of Computer Science, The University of Texas at Dallas, Richardson, TX 75080, USA. {abhirup.mukherjee, lkhan, bxt043000}@utdallas.edu

A. Activation Steering

Zou et al. [5] demonstrated that behavioral concepts lie on linear axes within the activation space of an LLM. Turner et al. [4] and Rimsky et al. [6] showed that injection of these concept vectors can steer LLM outputs even without any optimization. More recently, Rimsky et al. [6] introduced CAA for concept-based steering, and we build upon and adapt it to smart contracts. Additionally, Im and Li [9] provided theoretical evidence that the difference-of-means estimator is optimal for the extraction of concept vectors.

On the other hand, Wu et al. [10] showed that prompting is more effective than steering as long as the scale is large enough, and Bigelow et al. [11] view steering as the shifting of priors with diminishing returns in the presence of strong few-shot examples—both of which are consistent with our findings.

B. Usage of LLMs in Vulnerability Detection

At the higher-cost end, fine-tuned approaches like Smart-LLaMA-DPO [12] achieve a +10.43% boost in F1 score with the help of a three-stage post-training process for the LLaMA-3.1-8B model. These methods are computationally expensive, unlike activation steering. At the no-cost end, Alam et al. [13] performed a benchmarking study on several LLMs for vulnerabilities in Solidity.

Li et al. [7] present the only existing work that employs activation steering in code vulnerability detection, where they perform C/C++ binary classification on the SARD dataset and report a +16% improvement in F1 score. While they observed improvements across types, we find that our gain is limited to primarily one category. Ding et al. [14] showed that standard benchmarks tend to be overly optimistic because of data contamination, which motivated our discussion on the same.

III. METHODOLOGY

A. Problem Formulation

We pose smart contract vulnerability detection as an eleven-class classification problem, with the classes being the top 10 DASP vulnerability classes [8] and the SAFE class, and evaluate on the SmartBugs Curated dataset [8]. Using a leave-one-category-out prompting technique, we present the models with few-shot prompts that exclude all instances of the held-out category and test the model on its ability to detect the held-out category. Our primary research question aims to find whether activation steering improves detection of the held-out category beyond few-shot results.

Each prompt consists of a task instruction, k labeled source code examples with labels, and the test contract. The evaluation prompt includes one example from each of the non-held-out DASP categories and two examples from the SAFE category (typically $k = 11$). We select examples deterministically—the first contract in each category with source code containing fewer than 2,000 characters is always chosen to ensure reproducibility is maintained across runs.

B. Steering Vector Extraction

To detect the influence of the data source and the pairing strategy on the effectiveness of steering, we employ two methods to extract the steering vectors. For both methods, LLM activations are extracted at the last token position of the target layer ℓ :

$$h_i^\ell = f_\ell(x_i)_{-1} \in \mathbb{R}^d \quad (1)$$

where f_ℓ is the transformer output at layer ℓ and the subscript -1 denotes the last token position.

To select the optimal layer, we linearly probe the model by performing principal component analysis (PCA), followed by logistic regression with 5-fold stratified cross-validation and Area Under the Receiver Operating Characteristic curve (AUROC) on binary vulnerable/safe classes.

Method 1: Cross-Domain Mean Steering (CDM). Following Zou et al.’s difference-in-means method in [5], per-category steering vectors are calculated as follows:

$$v_c^\ell = \bar{h}_c^\ell - \bar{h}_{\text{safe}}^\ell = \frac{1}{|S_c|} \sum_{i \in S_c} h_i^\ell - \frac{1}{|S_{\text{safe}}|} \sum_{j \in S_{\text{safe}}} h_j^\ell \quad (2)$$

where S_c denotes a set of contracts from Slither category c , and S_{safe} is the safe set. We take the average of all vulnerable activations in a universal vector before subtracting the safe mean: $v_{\text{univ}}^\ell = \bar{h}_{\text{vuln}}^\ell - \bar{h}_{\text{safe}}^\ell$. For our data source, we use 1,200 Slither-audited contracts (200 single-label contracts across 5 categories and 200 safe) which are then fed into a prompt wrapper and chat template. These contracts cover 5 out of the 10 DASP categories that we evaluate on.

Method 2: Contrastive Pair Steering (CPS). Here, we follow Li et al. [7] and Rimsky et al. [6] in computing per-pair direction vectors:

$$D_i^\ell = h_{V_i}^\ell - h_{P_i}^\ell \quad (3)$$

where $h_{V_i}^\ell$ and $h_{P_i}^\ell$ refer to activations of a vulnerable contract and its minimally-patched safe counterpart. The category steering vector is:

$$v_c^\ell = \frac{1}{|S_c|} \sum_{i \in S_c} D_i^\ell \quad (4)$$

We prepare 173 pairs of vulnerable and patched contracts over all 10 DASP categories by applying regex-based minimal patches (e.g., `.call.value()` $\rightarrow$ `.transfer()` for reentrancy) to the contracts in the SmartBugs dataset. Next, we extract activations from the raw code input without prompt wrapping. We apply the CPS method to both Qwen2.5-Coder-32B and CodeLlama-34B.

CDM utilizes unpaired group means of cross-domain data, while CPS utilizes paired per-sample subtraction of domain-matched data. Equations (2) and (4) are mathematically equivalent if and only if the number of groups is the same and perfectly paired, which is not true in our case. By contrasting the two methods, we determine whether precision in the extraction mechanism has any influence on the efficacy of steering.

Algorithm 1 Steered Evaluation Pipeline

Require: Model M , test set $\mathcal{T}$, held-out type c^* , layer ℓ^* , vector v^{ℓ^*} , magnitude α

Ensure: Predictions $\hat{Y}_{\text{base}}$, $\hat{Y}_{\text{steer}}$

```
1: Build prompt  $P$ : one example per category in  $\mathcal{C} \setminus \{c^*\} + 2$ 
   SAFE
2: for each  $x_i \in \mathcal{T}$  do
3:    $\hat{y}_i^{\text{base}} \leftarrow M(P \oplus x_i)$  {Baseline}
4: end for
5: Attach hook:  $\tilde{h}^{\ell^*} \leftarrow h^{\ell^*} + \alpha \cdot v^{\ell^*}$ 
6: for each  $x_i \in \mathcal{T}$  do
7:    $\hat{y}_i^{\text{steer}} \leftarrow M(P \oplus x_i)$  {Steered}
8: end for
9: return  $\hat{Y}_{\text{base}}$ ,  $\hat{Y}_{\text{steer}}$ , McNemar( $\hat{Y}_{\text{base}}$ ,  $\hat{Y}_{\text{steer}}$ ,  $Y$ )
```

C. Steered Inference

During inference, we use the forward hook function in PyTorch to add the following modification to the hidden state for each forward pass through the target layer:

$$\tilde{h}^{\ell} = h^{\ell} + \alpha \cdot v^{\ell} \quad (5)$$

where $h^{\ell} \in \mathbb{R}^{B \times T \times d}$ is the target layer output, $v^{\ell} \in \mathbb{R}^d$ is the steering vector that is distributed over the entire batch and token dimension, and $\alpha \in \mathbb{R}$ is the scalar steering magnitude. By keeping the forward hook attached, we inject the steering vector at each generation step. We test the model with two types of vectors: the *universal* vector, where the same vector is injected irrespective of the vulnerability type, and the *category-specific* vector which is matched with the held-out vulnerability type.

We use greedy decoding for output generation and cap it at 100 new tokens. The evaluation pipeline is shown in Fig. 1 and Algorithm 1. Phase ① represents the few-shot prompt using the leave-one-category-out method explained in Section III-A. Phase ② represents the extraction of the steering vectors using the two methods (CDM and CPS) explained in Section III-B. Phase ③ is the steered inference using the injected steering vector at the target layer using Eq. (5).

D. Statistical Validation

For a paired comparison of our steered and baseline results, we use McNemar’s exact test [15]. This test constructs a 2×2 contingency table, where only the discordant pairs are taken into account, i.e., those pairs where the predictions for the two methods are correct and incorrect, respectively. We use the exact variant of this test since the number of discordant pairs is less than 15 for our dataset. For the best run on the ULL test set, the steering method corrects 11 predictions and introduces 1 new error, giving us a p -value of 0.006. We also report the results using the bootstrap 95% confidence interval with 10,000 iterations ($B = 10,000$) and Cohen’s h effect sizes [16]. Across all configurations, Cohen’s h stays at or below 0.18—under the 0.20 threshold for a small effect—which is consistent with a calibration-scale intervention as opposed to a change in reasoning. It is worth noting that since we search over 150 configurations across layers, magnitudes, vector types, and

held-out categories, this nominal $p = 0.006$ value is not corrected for multiple comparisons and should be interpreted as an exploratory result.

In order to attribute these effects to steering and to rule out noise, we use random direction control vectors. These have the same magnitude (same L_2 norm) and are almost orthogonal to the steering direction.

For the variance analysis of the stability of the baseline model, we used three random seeds (42, 123, 456) and different few-shot examples.

IV. EXPERIMENTS AND RESULTS

A. Experimental Setup

We evaluate the performance of six models: Qwen2.5-Coder-32B-Instruct (primary), CodeLlama-34B-Instruct, Gemma-2-9B, Llama-3.1-8B-Instruct, Granite-3.1-8B-Instruct, Mistral-7B-Instruct. Our held-out test set consists of 223 samples: 173 from SmartBugs Curated [8] across 10 DASP categories, along with 50 SAFE contracts from the Slither dataset. We hold out the following vulnerabilities: unchecked low-level calls (ULL, $n = 70$), reentrancy (REEN, $n = 33$), access control (AC, $n = 21$), arithmetic (ARITH, $n = 21$). Since the dataset is long-tailed (i.e. the other six DASP categories contain fewer than 13 contracts each), we address this with a robustness analysis in Section IV-F.

For both methods, we tested 116 configurations for Qwen (80 CDM + 36 CPS), 52 configurations for CodeLlama (16 CDM + 36 CPS), and random direction controls for 3 layers for each remaining model, all over four held-out types and present our best results. We selected these configurations post-hoc based on their test-set accuracy, hence our reported gains represent the best-case scenario. All experiments were run on Google Colab on NVIDIA RTX 6000 Pro Blackwell GPUs with 96GB of GDDR7 VRAM.

B. Steering Results

In Table I, we show the best results for Qwen, using CDM vectors across all four held-out vulnerabilities. Accuracy results show the classification accuracy (%), while the F1 results show the held-out-type F1 score, which combines precision and recall on the held-out vulnerability type. ULL is the only held-out type for which the results are statistically significant (nominal $p = 0.006$), with a bootstrap 95% confidence interval of $[+1.8\text{pp}, +7.6\text{pp}]$. The best configuration has 83.9% accuracy (ULL), while the worst has 78.0% (ARITH). For the other held-out types, accuracy ranges between 79.4% and 80.3%. For Qwen, random direction controls are always either neutral or harmful, and this fact supports our claim on directional specificity of steering vectors.

Fig. 2 visualizes these results. There is a pronounced gap between steered and baseline numbers for ULL, while the other three categories show small or negligible differences. ULL is the only category for which accuracy and F1 score improve under steering. For ARITH, accuracy improves but F1 score worsens since held-out recall drops from 90.5% to

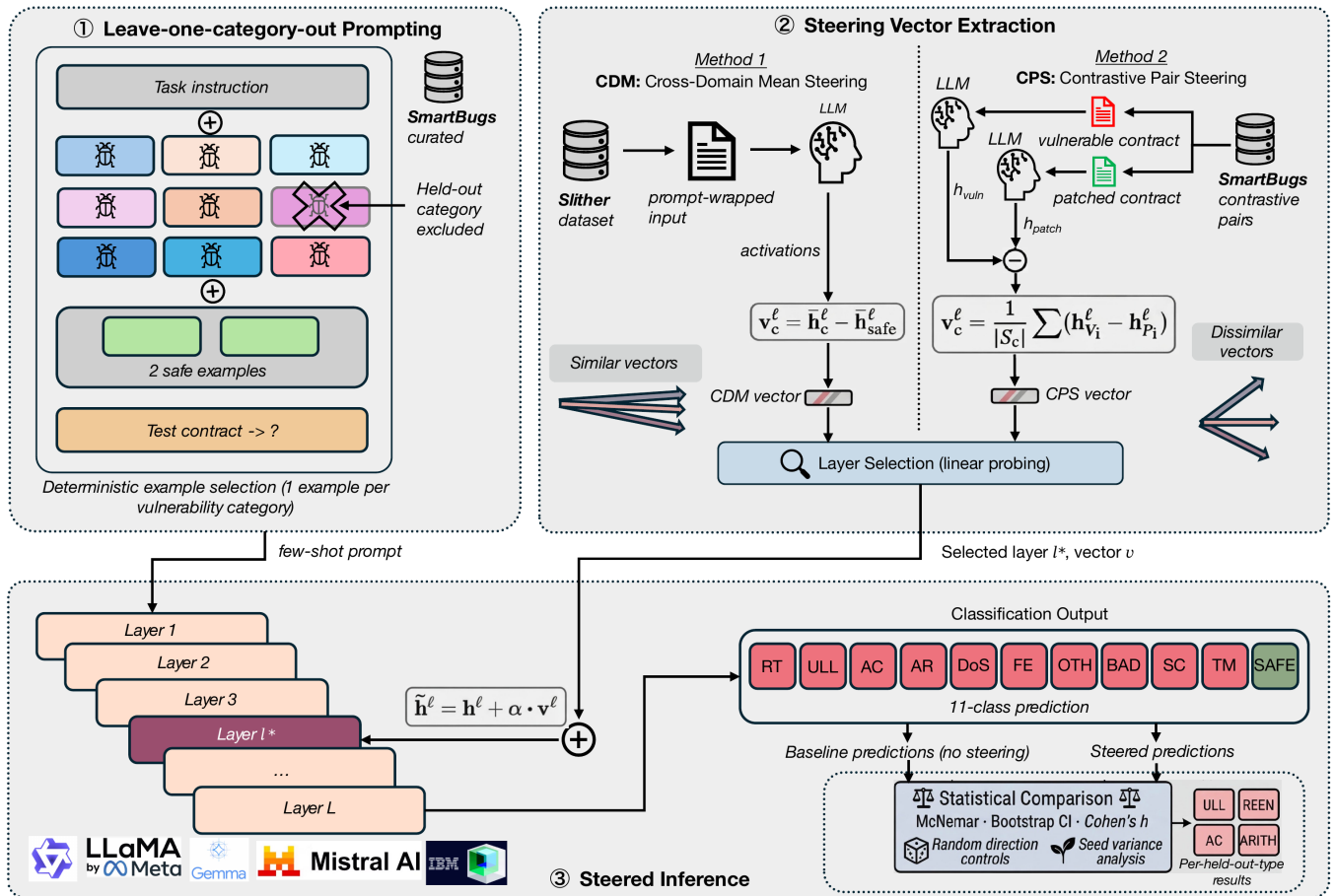


Fig. 1. Pipeline overview showing three phases. Phase ① constructs a leave-one-category-out few-shot prompt that excludes the held-out vulnerability type. Phase ② extracts steering vectors via two methods—CDM (cross-domain group means) and CPS (domain-matched contrastive pairs). Phase ③ performs steered inference by injecting the selected vector at the target layer.

TABLE I

STEERING RESULTS: BASELINE VS. BEST STEERED VS. RANDOM DIRECTION (QWEN + CDM, $n = 223$). ACC: OVERALL ACCURACY (%). F1: HELD-OUT-TYPE F1. STEERED: BEST CONFIG PER TYPE. RANDOM: L16 $\alpha = 2.0$ FOR ALL TYPES.

HO	Acc (%)			F1			Δ_S	p
	Base	Steer	Rand	Base	Steer	Rand		
ULL	79.4	83.9	78.5	.855	.904	.837	+4.5	.006
REEN	78.0	80.3	76.7	.741	.811	.779	+2.2	.125
AC	77.6	79.4	76.2	.585	.579	.585	+1.8	.219
ARITH	76.7	78.0	74.4	.950	.895	.950	+1.3	.453

81.0%. ULL has the highest number of data points ($n = 70$) and largest room for improvement (80.0% recall for ULL versus 90.5%/90.9% for REEN/ARITH), which supports the theory that steering effects are strongest where in-context evidence is weakest [11].

C. Cross-Model Results

Table II summarizes baseline and steering results for all six models on ULL held-out data. Qwen2.5-Coder-32B leads with the highest accuracy among all models at 79.4%, followed by Gemma-2-9B at 75.8%, in spite of being 3.5× smaller. CodeLlama-34B, ranked second in terms of param-

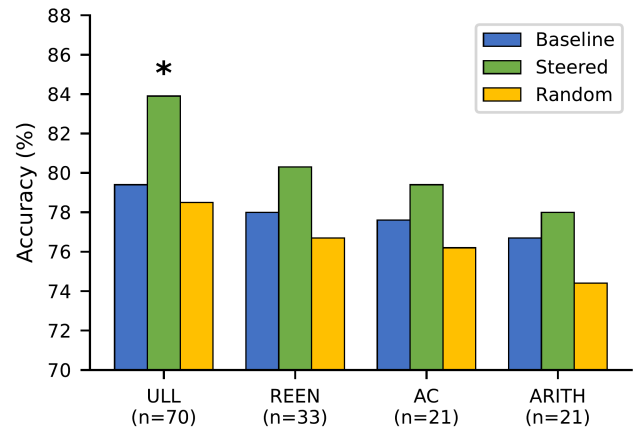


Fig. 2. Accuracy by held-out type: baseline vs. steered vs. random direction (Qwen + CDM vectors). * denotes $p < 0.01$.

eters, only achieves an accuracy of 39.5%, demonstrating how architecture and pre-training matter more than model size for understanding Solidity semantics; independent evaluations also report little evidence of Solidity in CodeLlama’s pre-training [17]. Three models show positive steering effects: Qwen (+4.5pp, $p = 0.006$), CodeLlama (+9.0pp, $p =$

TABLE II

CROSS-MODEL RESULTS ON ULL HELD-OUT ($n = 223$). Δ_S : STEERING IMPROVEMENT. Δ_R : RANDOM DIRECTION CHANGE.

Model	Params	Base	Steered	Δ_S	Δ_R	p
Qwen-32B	32B	79.4	83.9	+4.5	−0.9	0.006
Gemma-9B	9B	75.8	75.8	+0.0	−4.0	—
Llama-8B	8B	62.8	61.0	−1.8	+6.7	—
Granite-8B	8B	56.1	59.2	+3.1	−11.2	0.296
Mistral-7B	7B	45.3	53.4	+8.1	−13.5	0.018
CL-34B	34B	39.5	48.4	+9.0	−2.2	0.004

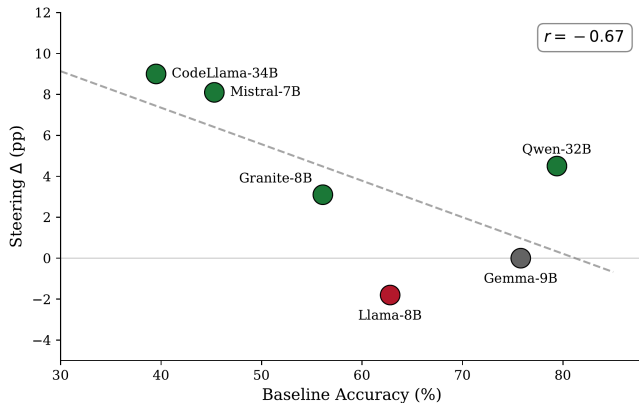


Fig. 3. Pre-training ceiling: baseline accuracy vs. steering improvement across six models. Green = positive effect, gray = neutral, red = anomalous (random outperforms real). Dashed line: linear fit ($r = -0.67$).

0.004), and Mistral (+8.1pp, $p = 0.018$). Four out of six models show directional specificity.

Interestingly, we note that Gemma-2-9B, the second strongest model in terms of accuracy, has marginal steering gains, while the weakest models (CodeLlama, Mistral) show the largest absolute gains. The Pearson correlation delta between baseline accuracy and steering is $r = -0.67$ —this suggests, although not significantly at six models ($p \approx 0.14$), that stronger models benefit less from steering. Models close to their knowledge ceiling (like Qwen at 79.4%) improve little, while weaker models (like CodeLlama at 39.5%, Mistral at 45.3%) improve by a more significant amount.

The Llama-3.1-8B model was an exception since the random direction vector produced a better (+6.7pp) improvement than the actual steering vector (−1.8pp). It also classified contracts as UNKNOWN 12 times during its baseline test, indicating unstable classification behavior that does not mix well with activation steering.

D. Lightweight Intervention Comparison

In Table III, we compare the performance of three lightweight methods on the ULL held-out classification task. Steering is the only such method that improves over the few-shot baseline on all metrics.

E. CDM vs. CPS Extraction

While CPS is methodologically more sound since it uses domain-matched pairs with nearly orthogonal category vectors (cosine similarity of 0.06–0.09), it proves to be less

TABLE III

LIGHTWEIGHT INTERVENTIONS ON ULL HELD-OUT ($n = 223$). PRECISION AND F1 ARE MACRO-AVERAGED ACROSS ALL 11 CLASSES.

Intervention	Acc	Prec	F1	ULL Rec	ΔAcc
Zero-shot	50.7	47.9	38.8	48.6	−28.7
Few-shot	79.4	69.9	62.7	80.0	—
Few-shot + Steer	83.9	71.1	63.5	87.1	+4.5

effective than CDM’s highly correlated vectors (cosine similarity of 0.75–0.91). We deduce that steering acts as a calibration mechanism that adjusts the model’s safe/vulnerable detection threshold and does not enhance vulnerability reasoning. Given single-layer access, orthogonal transformations of a candidate steering direction yield indistinguishable outputs, and hence that vector cannot be uniquely identified [18].

F. Robustness to Class Imbalance

Six out of the eleven classes in the SmartBugs Curated dataset have less than 13 contracts each. To ensure that the observed steering gains were not merely due to the presence of the sparse classes, we repeat the same evaluation only for those classes that have more than 13 contracts (ULL, REEN, AC, ARITH, SAFE) using both cross-domain (CDM) and domain-matched augmented (CPS) methods. In doing so, we notice the following improvements: ULL, REEN, AC, ARITH saw increases of $\{+5.2, +2.6, +2.1, +1.6\}$ pp respectively for CDM and $\{+1.4, +0.7, +2.1, +0.7\}$ pp respectively for CPS.

V. DISCUSSION

Pre-training ceiling. As one can see in Fig. 3, baseline accuracy and steering improvement (Δ_S) appear to be negatively correlated ($r = -0.67$). Additionally, a coarser extraction technique outperformed a finer one in our experiments. These outcomes suggest two things: firstly, stronger models might benefit less from steering, and secondly, there exists a pre-training knowledge boundary that steering fails to overcome. Few-shot prompting provides strong evidence for activating the existing vulnerability knowledge of the model [11], and steering activates this knowledge as well. Activation steering does not, in our experiments, introduce new concepts of vulnerability to large language models. This is further supported by AxBench’s findings, where prompting was found to be superior when compared to steering at scale [10]. Operationally, this favors steering as a lightweight calibration technique on top of an already-strong model, with static analyzers or fine-tuned models being more suitable for higher recall on novel vulnerability types.

Access control as non-linear representation. For all of our lightweight intervention techniques, AC recall remains unchanged at 57.1% for all few-shot baselines and for 18 out of the 20 steering configurations. The unchanging nature of this result across different techniques strongly suggests that this particular vulnerability type may not be represented in linear activation space, thereby making it resistant to perturbations by linear steering vectors [19].

Dataset contamination. Our best performing model, Qwen2.5-Coder-32B was trained on 5.5 trillion tokens of data gathered from public GitHub repositories. Since Solidity is one of the explicitly supported languages by this model [20], we suspect that the SmartBugs Curated dataset likely exists in its training data. LessLeak-Bench [21] reported that 100% of QuixBugs and 55.7% of BigCloneBench have been exposed to LLM training data; the SmartBugs dataset may share a similar exposure risk. Validation against post-cutoff datasets, such as FORGE [22], is critical as future work.

Limitations. Linear additive steering was the only method we tested in this work. While we conclude that steering acts as a confidence threshold adjustment in linear activation space, it may yet produce stronger results under projection-based/gradient-based techniques. Additionally, the Llama-3.1-8B result indicates that directional specificity may not universally hold across all architectures. Although plausible, our calibration interpretation is inferred indirectly, and direct evidence such as confidence-calibration plots and confusion matrices remain future work.

VI. CONCLUSION

We demonstrate the first application of activation steering for smart contract security, using six models across five different architectural families to test the effectiveness of lightweight intervention techniques. Among these, steering is the only one that is beneficial. We also study the effects of activation steering using a coarse and a fine method for extracting the vector. The coarse method performs better than the finer one, and our study suggests that activation steering functions not as an improvement in vulnerability representations within the model, but as an adjustment to its decision threshold. A pre-training knowledge ceiling limits how beneficial all lightweight interventions are, with stronger models getting diminishing returns from activation steering.

REFERENCES

- [1] “DEFI Hacks & Exploits Database - DefiLlama,” <https://defillama.com/hacks>, accessed: 2026-04-19.
- [2] J. Feist, G. Grieco, and A. Groce, “Slither: A Static Analysis Framework for Smart Contracts,” in *2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*. Montreal, QC, Canada: IEEE, May 2019, pp. 8–15. [Online]. Available: <https://ieeexplore.ieee.org/document/8823898/>
- [3] M. N. Sakib and L. Khan, “VulnPatch: Multi-Agent Automated Smart Contract Vulnerability Detection, Explanation, and Mitigation Framework,” in *2025 7th International Conference on Blockchain Computing and Applications (BCCA)*. Durbovnic, Croatia: IEEE, Oct. 2025, pp. 301–309. [Online]. Available: <https://ieeexplore.ieee.org/document/11229600/>
- [4] A. M. Turner, L. Thiergart, G. Leech, D. Udell, J. J. Vazquez, U. Mini, and M. MacDiarmid, “Steering Language Models With Activation Engineering,” 2023. [Online]. Available: <https://arxiv.org/abs/2308.10248>
- [5] A. Zou, L. Phan, S. Chen, J. Campbell, P. Guo, R. Ren, A. Pan, X. Yin, M. Mazeika, A.-K. Dombrowski, S. Goel, N. Li, M. J. Byun, Z. Wang, A. Mallen, S. Basart, S. Koyejo, D. Song, M. Fredrikson, J. Z. Kolter, and D. Hendrycks, “Representation Engineering: A Top-Down Approach to AI Transparency,” 2023. [Online]. Available: <https://arxiv.org/abs/2310.01405>
- [6] N. Rimskey, N. Gabrieli, J. Schulz, M. Tong, E. Hubinger, and A. Turner, “Steering Llama 2 via Contrastive Activation Addition,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Bangkok, Thailand: Association for Computational Linguistics, 2024, pp. 15 504–15 522. [Online]. Available: <https://aclanthology.org/2024.acl-long.828>
- [7] J. Li, L. Cui, J. Zhang, H. Fei, Y. Chen, and H. Zhu, “Steering Large Language Models for Vulnerability Detection,” in *ICASSP 2025 - 2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. Hyderabad, India: IEEE, Apr. 2025, pp. 1–5. [Online]. Available: <https://ieeexplore.ieee.org/document/10887736/>
- [8] J. F. Ferreira, P. Cruz, T. Durieux, and R. Abreu, “SmartBugs: a framework to analyze solidity smart contracts,” in *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*. Virtual Event Australia: ACM, Dec. 2020, pp. 1349–1352. [Online]. Available: <https://dl.acm.org/doi/10.1145/3324884.3415298>
- [9] S. Im and S. Li, “A Unified Understanding and Evaluation of Steering Methods,” 2025. [Online]. Available: <https://arxiv.org/abs/2502.02716>
- [10] Z. Wu, A. Arora, A. Geiger, Z. Wang, J. Huang, D. Jurafsky, C. D. Manning, and C. Potts, “AxBench: Steering LLMs? Even Simple Baselines Outperform Sparse Autoencoders,” 2025. [Online]. Available: <https://arxiv.org/abs/2501.17148>
- [11] E. Bigelow, D. Wurgaft, Y. Wang, N. Goodman, T. Ullman, H. Tanaka, and E. S. Lubana, “Belief Dynamics Reveal the Dual Nature of In-Context Learning and Activation Steering,” 2025. [Online]. Available: <https://arxiv.org/abs/2511.00617>
- [12] L. Yu, Z. Huang, H. Yuan, S. Cheng, L. Yang, F. Zhang, C. Shen, J. Ma, J. Zhang, J. Lu, and C. Zuo, “Smart-LLaMA-DPO: Reinforced Large Language Model for Explainable Smart Contract Vulnerability Detection,” 2025. [Online]. Available: <https://arxiv.org/abs/2506.18245>
- [13] M. T. Alam, R. Halder, and A. Maiti, “Detection Made Easy: Potentials of Large Language Models for Solidity Vulnerabilities,” 2024. [Online]. Available: <https://arxiv.org/abs/2409.10574>
- [14] Y. Ding, Y. Fu, O. Ibrahim, C. Sitawarin, X. Chen, B. Alomair, D. Wagner, B. Ray, and Y. Chen, “Vulnerability Detection with Code Language Models: How Far are We?” in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. Ottawa, ON, Canada: IEEE, Apr. 2025, pp. 1729–1741. [Online]. Available: <https://ieeexplore.ieee.org/document/11029911/>
- [15] Q. McNemar, “Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages,” *Psychometrika*, vol. 12, no. 2, pp. 153–157, June 1947. [Online]. Available: https://www.cambridge.org/core/product/identifier/S0033312300045178/type/journal_article
- [16] J. Cohen, *Statistical power analysis for the behavioral sciences*. routledge, 2013. [Online]. Available: <https://api.taylorfrancis.com/content/books/mono/download?identifierName=doi&identifierValue=10.4324/9780203771587&type=googlepdf>
- [17] A. Dinaburg, “Evaluating Solidity support in AI coding assistants,” Nov. 2024. [Online]. Available: <https://blog.trailofbits.com/2024/11/19/evaluating-solidity-support-in-ai-coding-assistants/>
- [18] S. Venkatesh and A. M. Kurupath, “On the Non-Identifiability of Steering Vectors in Large Language Models,” Apr. 2026. [Online]. Available: <http://arxiv.org/abs/2602.06801>
- [19] J. Braun, “Understanding Unreliability of Steering Vectors in Language Models: Geometric Predictors and the Limits of Linear Approximations,” 2026. [Online]. Available: <https://arxiv.org/abs/2602.17881>
- [20] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu, K. Lu, K. Dang, Y. Fan, Y. Zhang, A. Yang, R. Men, F. Huang, B. Zheng, Y. Miao, S. Quan, Y. Feng, X. Ren, X. Ren, J. Zhou, and J. Lin, “Qwen2.5-Coder Technical Report,” 2024. [Online]. Available: <https://arxiv.org/abs/2409.12186>
- [21] X. Zhou, M. Weyssow, R. Widyasari, T. Zhang, J. He, Y. Lyu, J. Chang, B. Zhang, D. Huang, and D. Lo, “LessLeak-Bench: A First Investigation of Data Leakage in LLMs Across 83 Software Engineering Benchmarks,” 2025. [Online]. Available: <https://arxiv.org/abs/2502.06215>
- [22] J. Chen, Y. Shen, J. Zhang, Z. Li, J. Grundy, Z. Shao, Y. Wang, J. Wang, T. Chen, and Z. Zheng, “FORGE: An LLM-driven Framework for Large-Scale Smart Contract Vulnerability Dataset Construction,” 2025. [Online]. Available: <https://arxiv.org/abs/2506.18795>